

8. Introduction

If you have spent time with SQL, you have almost certainly been *reading* data with `SELECT`. That is the comfortable half of the language—nothing you type can hurt anything. But real applications do not just read data. They add new customers, correct wrong values, and remove records that no longer belong. The moment you start *writing* data, SQL stops being a safe reading tool and becomes something with a sharp edge.

That sharp edge has a name: the **missing `WHERE` clause**. It is the single most common way engineers accidentally destroy production data, and it takes about a tenth of a second to do it. Interviewers love this topic precisely because it separates people who have *used* a database from people who have only read about one. Anyone can memorize the syntax of `UPDATE`. Far fewer can explain *why* one forgotten keyword rewrites two thousand rows instead of one.

In this lesson we cover the three commands that make up **Data Manipulation Language (DML)**: `INSERT`, `UPDATE`, and `DELETE`. You will learn exactly when to reach for each one, why two of them are dangerous, and the professional habits—`SELECT`-first, transactions, and soft deletes—that let senior engineers modify live data without fear. This is core knowledge for backend development, data engineering, and any software engineering role that touches a relational database.

9. Problem Overview

Every relational database is built on a small set of verbs. Reading is one. Changing is the other.

DML—Data Manipulation Language—is the group of SQL commands that manipulate the actual data stored inside your tables. If `SELECT` is reading a book, DML is picking up a pen and editing it. Stripped of jargon, DML is just three actions:

Command	What it does	Danger level
<code>INSERT</code>	Adds a new row	Low
<code>UPDATE</code>	Changes existing rows	High
<code>DELETE</code>	Removes whole rows	High

The core challenge is not memorizing syntax—it is understanding **scope**. `UPDATE` and `DELETE` both accept a `WHERE` clause that decides *which* rows are affected. Get the `WHERE` right and you touch exactly the rows you intended. Omit it, and SQL applies your change to **every single row in the table**, silently and completely. The database does exactly what you *said*, not what you *meant*.

10. Example

Let's work with a tiny `users` table all lesson. Small on purpose, so you can hold the whole thing in your head.

Starting state:

id name age

1	Alice	30
2	Bob	25

Two people. Now watch how each command grows it, tweaks it, or shrinks it.

INSERT — add Carol:

```
INSERT INTO users (id, name, age)
VALUES (3, 'Carol', 28);
```

Result:

id name age

1	Alice	30
2	Bob	25
3	Carol	28

One new row appears at the bottom. The values landed under the right columns because we listed the columns first (`id, name, age`) and lined the values up in the same order.

UPDATE — fix Alice's age:

```
UPDATE users
SET age = 31
WHERE id = 1;
```

Result:

id name age

1	Alice	31
2	Bob	25
3	Carol	28

Only Alice changed, because `WHERE id = 1` pointed at exactly one row.

DELETE — remove Bob:

```
DELETE FROM users  
WHERE id = 2;
```

Result:

id name age

1 Alice 31

3 Carol 28

Bob is gone; Alice and Carol are untouched. The `WHERE` pulled out the one matching row and left the rest alone.

11. Intuition

Here is how an experienced engineer thinks about these commands, before writing a single line.

First, picture the table as a spreadsheet your customers depend on. Every DML command is a physical action on that sheet: `INSERT` staples a new row to the bottom, `UPDATE` reaches into existing rows and rewrites cells, `DELETE` tears whole rows out. Once you see it physically, the danger becomes obvious. Adding a row rarely disturbs what was already there—that's why `INSERT` is the calm one. But rewriting and tearing out *act on rows that already matter*.

Second, understand what `WHERE` really is. It is not part of "finding" data—it is a **filter that defines scope**. When you write `UPDATE users SET age = 31`, SQL reads that as: "for the set of rows I'm about to touch, set age to 31." The `WHERE` clause is the only thing that shrinks "the set of rows" from *all of them* down to *the ones you want*. Remove the filter and the set defaults back to everything. SQL isn't being cruel; it's being literal.

Third—and this is the mental shift that saves careers—**treat the `WHERE` clause as the real command and `UPDATE` / `DELETE` as just the action attached to it**. A senior engineer writes the `WHERE` first, verifies it, and only then decides what to do with the rows it selects. That reordering is the whole game.

12. Brute Force Solution (the naive, dangerous way)

Idea: Just write the command and run it. You know what you want to change, so you type the `UPDATE` or `DELETE`, hit execute, and trust yourself to include the `WHERE`.

Algorithm: 1. Decide which rows to change. 2. Write the DML statement with a `WHERE` clause. 3. Run it directly against the table.

Advantages: - Fast—one statement, one execution. - Simple to type when you're confident.

Disadvantages: - **No safety net.** If you forget the `WHERE`, or your `WHERE` is broader than you thought, the change is instant and irreversible. - A wrong `UPDATE` produces bad *values*, not errors—so nothing warns you. You may not notice for weeks. - Impossible to preview what will be affected before it happens.

Complexity of the mistake: The failure isn't algorithmic—it's operational. Running `UPDATE users SET age = 31;` with no `WHERE` rewrites **all N rows** in a single statement. On a live table that's thousands of records corrupted in roughly one tenth of a second, with no undo.

13. Optimal Solution (the safe engineering workflow)

Professionals don't rely on typing carefully. They build a workflow that makes the dangerous case *hard to trigger*. Three layers, from cheapest to strongest.

Layer 1 — SELECT-first (the five-second habit)

Before any `UPDATE` or `DELETE`, take your exact `WHERE` clause and run it as a `SELECT *` first:

```
SELECT * FROM users WHERE id = 1;
```

Whatever rows come back are *exactly* the rows your `UPDATE` or `DELETE` will hit. If two rows return, two rows change. If zero return, your filter is wrong. It costs five seconds and it has never once let a careful engineer down.

Layer 2 — Transactions (draft mode)

A transaction is like editing in draft. You `BEGIN`, make your changes, inspect the result, and only then decide to keep or discard them:

```
BEGIN;

UPDATE users SET age = 31 WHERE id = 1;

SELECT * FROM users;  -- inspect the result

COMMIT;  -- keep it, OR:
-- ROLLBACK;  -- undo everything since BEGIN
```

`COMMIT` makes the changes permanent. `ROLLBACK` erases them as if they never happened. This is your undo button for the database.

Layer 3 — Soft delete (never truly lose data)

Instead of physically removing a row, flip a flag:

```
-- Schema has an is_deleted column
UPDATE users SET is_deleted = TRUE WHERE id = 2;

-- Normal reads just filter it out
SELECT * FROM users WHERE is_deleted = FALSE;
```

The row still exists; it's just hidden. The minus is a little extra complexity in every query. The plus is that nothing is ever permanently lost, and you can sleep at night.

Decision table:

Situation	Reach for
Adding a brand-new record	INSERT
Correcting a value in existing rows	UPDATE (with WHERE)
Removing a row entirely	DELETE (with WHERE)
Blanking out one field only	UPDATE ... SET col = NULL (not DELETE)
Any risky edit on live data	Wrap in BEGIN ... COMMIT/ROLLBACK
Data you might need to recover	Soft delete flag

14. Python Code

Here's the complete workflow demonstrated in Python using the standard-library `sqlite3` module—no external dependencies. It shows the safe SELECT-first-then-modify pattern inside a transaction.

```
import sqlite3

def build_users_table(conn: sqlite3.Connection) -> None:
    """Create a tiny users table and seed it with two rows."""
    conn.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")
    conn.executemany(
        "INSERT INTO users (id, name, age) VALUES (?, ?, ?)",
        [(1, "Alice", 30), (2, "Bob", 25)],
    )

def preview(conn: sqlite3.Connection, where: str, params: tuple) -> list:
    """SELECT-first: return the exact rows a WHERE clause will affect."""
    rows = conn.execute(f"SELECT * FROM users WHERE {where}", params).fetchall()
    print(f"WHERE {where} matches {len(rows)} row(s): {rows}")
    return rows
```

```

def main() -> None:
    # In-memory DB so the demo leaves nothing behind.
    conn = sqlite3.connect(":memory:")
    build_users_table(conn)

    # INSERT: add a new row.
    conn.execute("INSERT INTO users (id, name, age) VALUES (?, ?, ?)", (3, "Carol", 28))

    # Transaction as draft mode: change, inspect, then commit or roll back.
    try:
        # SELECT-first BEFORE the risky UPDATE.
        if preview(conn, "id = ?", (1,)):
            conn.execute("UPDATE users SET age = ? WHERE id = ?", (31, 1))

        # DELETE, again scoped by a verified WHERE.
        if preview(conn, "id = ?", (2,)):
            conn.execute("DELETE FROM users WHERE id = ?", (2,))

        conn.commit() # keep the changes
    except Exception:
        conn.rollback() # undo everything on any error
        raise

    print("Final:", conn.execute("SELECT * FROM users ORDER BY id").fetchall())
    conn.close()

if __name__ == "__main__":
    main()

```

15. Code Walkthrough

- **build_users_table** creates the **users** table and uses **executemany** to seed Alice and Bob in one call—cleaner than two separate inserts.
- **preview** is the SELECT-first habit turned into a function. It runs your **WHERE** clause as a read-only **SELECT**, prints how many rows match, and returns them. Every risky command is gated behind this.
- **Parameterized queries (? placeholders)** appear everywhere. We never glue user values into SQL strings—that prevents SQL injection and is the professional default. (Note the **WHERE** *structure* in **preview** is interpolated, but the *values* are always parameters.)
- **The INSERT** lists columns explicitly (**id, name, age**). Always name your columns, even though SQL lets you skip them—it makes the statement survive future schema changes.

- **The try/except** wraps both risky operations. `conn.commit()` only runs if everything succeeded; any exception triggers `conn.rollback()`, restoring the table to its state at the last commit. This is the transaction "draft mode" in code.
- **The UPDATE and DELETE** each carry a `WHERE id = ?`, so they touch exactly one verified row.

16. Dry Run

Let's trace `main()` step by step.

1. **Start:** table is `[(1, Alice, 30), (2, Bob, 25)]`.
2. **INSERT Carol:** table becomes `[(1, Alice, 30), (2, Bob, 25), (3, Carol, 28)]`.
3. **preview("id = ?", (1,))** runs `SELECT * FROM users WHERE id = 1`. Prints: `matches 1 row(s): [(1, 'Alice', 30)]`. One row—safe to proceed.
4. **UPDATE age = 31 WHERE id = 1:** Alice's age becomes 31. Table: `[(1, Alice, 31), (2, Bob, 25), (3, Carol, 28)]`.
5. **preview("id = ?", (2,))** runs `SELECT * FROM users WHERE id = 2`. Prints: `matches 1 row(s): [(2, 'Bob', 25)]`. One row confirmed.
6. **DELETE WHERE id = 2:** Bob's row is removed. Table: `[(1, Alice, 31), (3, Carol, 28)]`.
7. **commit():** changes are made permanent.
8. **Final output:** `[(1, 'Alice', 31), (3, 'Carol', 28)]`.

Notice how each mutation was previewed first, so at no point were we guessing about scope.

17. Complexity Analysis

- **INSERT of one row: $O(1)$** time on average—appending a single row. (Index maintenance adds a small $O(\log N)$ cost per index, but it's not proportional to table size.)
- **UPDATE / DELETE with a WHERE on a primary key: $O(\log N)$** time, because the database uses the index to jump straight to the matching row instead of scanning.
- **UPDATE / DELETE without a WHERE: $O(N)$** —it touches every one of the N rows. This is exactly why the missing `WHERE` is catastrophic: it turns a targeted $O(\log N)$ operation into a full-table $O(N)$ rewrite.
- **Space:** all three are **$O(1)$** extra space for the operation itself. Inside a transaction, the database keeps enough undo information to support `ROLLBACK`, which is proportional to the number of changed rows—another reason to keep your `WHERE` tight.

18. Alternative Solutions

UPSERT (INSERT ... ON CONFLICT). When you want to insert a row *or* update it if it already exists, doing a `SELECT` then branching to `INSERT / UPDATE` is two round trips and a race condition. Most databases offer a single atomic statement:

```
INSERT INTO users (id, name, age)
VALUES (1, 'Alice', 31)
ON CONFLICT (id) DO UPDATE SET age = EXCLUDED.age;
```

This is cleaner and safer than manual check-then-write. Use it whenever "insert or update" is the real intent.

TRUNCATE vs DELETE. If you genuinely want to empty an entire table, `TRUNCATE TABLE users` is far faster than `DELETE FROM users` because it doesn't process rows individually. But it's also less recoverable and often can't be rolled back—so it's a specialist tool, not a default.

19. Edge Cases

- **Empty table:** `UPDATE / DELETE` with a `WHERE` that matches nothing simply affects zero rows—no error. Good to confirm via `SELECT`-first.
- **WHERE matching duplicates:** if `WHERE age = 30` matches three people, all three change. This is the silent trap—always preview the count.
- **Blanking one field:** `DELETE` removes whole rows, never single columns. To clear one field, use `UPDATE users SET age = NULL WHERE id = 1`.
- **Missing WHERE entirely:** affects all N rows. The core disaster of the lesson.
- **NULL in the WHERE :** `WHERE age = NULL` matches *nothing*—`NULL` isn't equal to anything. Use `WHERE age IS NULL`.
- **Inserting a duplicate primary key:** fails with a constraint violation; the row is rejected, not silently overwritten.

20. Common Mistakes

1. **Forgetting the WHERE clause.** The headline mistake. `UPDATE users SET age = 31;` rewrites everyone. Always write the `WHERE` first.
2. **Trusting = with NULL.** `WHERE col = NULL` never matches. Use `IS NULL / IS NOT NULL`.

3. **Using DELETE to clear a single field.** DELETE removes rows. To empty a column, UPDATE ... SET col = NULL .
4. **Skipping column names in INSERT .** INSERT INTO users VALUES (3, 'Carol', 28) breaks silently if the table's column order changes. List columns explicitly.
5. **Not previewing with SELECT first.** Assuming your WHERE matches one row when it actually matches many.
6. **Running risky edits outside a transaction.** Without BEGIN / ROLLBACK , there's no undo when you realize the mistake.

21. Interview Questions

- **What happens if you run an UPDATE without a WHERE clause?** Every row in the table is updated to the new value.
- **How would you safely delete rows in a production database?** SELECT-first to verify scope, wrap in a transaction, and consider soft deletes for recoverability.
- **What's the difference between DELETE , TRUNCATE , and DROP ?** DELETE removes selected rows (row by row, rollback-able); TRUNCATE empties the whole table fast; DROP removes the table itself.
- **Why is a bad UPDATE more dangerous than a bad DELETE ?** A missing row is obvious; a wrong value looks like valid data and can hide for weeks.
- **How do you insert a row or update it if it already exists in one statement?** Use UPSERT (INSERT ... ON CONFLICT DO UPDATE).
- **What is a soft delete and when would you use it?** Flagging a row as deleted instead of removing it, used when you may need to recover or audit data.

22. Similar Problems

While these are SQL concepts rather than LeetCode algorithm puzzles, the SQL problem set on LeetCode exercises the same muscles:

- **LeetCode 627 – Swap Salary** (UPDATE with a CASE): direct practice of scoped updates.
- **LeetCode 196 – Delete Duplicate Emails** (DELETE): forces you to write a precise WHERE so you remove duplicates without touching originals—exactly the scope discipline from this lesson.
- **LeetCode 1667 – Fix Names in a Table** (UPDATE / SELECT): transforming existing values in place.

They're related because each one lives or dies on getting the WHERE /filter scope exactly right.

23. Key Takeaways

- **DML is three verbs:** `INSERT` (add), `UPDATE` (change), `DELETE` (remove).
- **`INSERT` is the gentle one;** `UPDATE` and `DELETE` share one deadly edge—the missing `WHERE` , which applies your change to every row.
- **Write the `WHERE` first.** Treat it as the real command.
- **`SELECT-first`** turns your `WHERE` into a five-second preview.
- **Transactions** give you an undo button; **soft deletes** mean nothing is ever truly lost.
- **A bad `UPDATE` is scarier than a bad `DELETE` ,** because wrong values hide.

24. Watch the Video

Reading the workflow is one thing—*seeing* rows appear, change, and vanish in real time makes it click. Watch the full walkthrough here, where we build the `users` table live and run each command step by step, including a look at exactly what the screen shows the instant a `WHERE` clause goes missing: [{LINK}](#)

25. About the Series

This lesson is part of **Fun with Learning Technology**, a series that turns intimidating engineering topics into small, confident steps. We keep tables tiny, examples concrete, and explanations honest about the real dangers you'll meet on the job—so you learn not just the syntax, but the *judgment* that separates a junior from a senior engineer. SQL, Python, algorithms, and the everyday craft of software engineering, one approachable lesson at a time.

26. Call To Action

If this made `INSERT` , `UPDATE` , and `DELETE` feel less scary, **like the video on YouTube**—it genuinely helps more developers find it. **Subscribe** so you don't miss the next SQL lesson, and **subscribe to the Substack** for the written deep-dives like this one. Got a "that edge case broke me" story of your own? **Drop it in the comments**, and **share this with a teammate** who's one missing `WHERE` away from a very bad afternoon.

Quick Review

When you need to add, change, or remove table rows, which SQL commands?

DML trio: INSERT adds new rows, UPDATE modifies existing rows, DELETE removes rows. DML = Data Manipulation Language, which changes stored data, not structure.

Time cost of UPDATE/DELETE with a WHERE on an unindexed column?

$O(n)$ — the engine scans every row to test the condition. An index on the filtered column drops it toward $O(\log n)$ lookup.

Extra space required by a basic INSERT, UPDATE, or DELETE?

$O(1)$ auxiliary per row for the operation itself; storage grows/shrinks by the row's size. Transaction logs/undo add temporary space until commit.

The scariest DML bug involving a missing clause?

Omitting WHERE: UPDATE or DELETE then hits every row in the table. Always test the filter with SELECT first, and wrap in a transaction.

UPDATE vs DELETE-then-INSERT for changing a row?

UPDATE edits in place, keeping the key and untouched columns. DELETE+INSERT loses old values, can break foreign keys, and reshuffles auto-IDs. Prefer UPDATE.

Follow-up: how do you undo a wrong UPDATE you just ran?

If still in an open transaction, ROLLBACK. Once committed, restore from backup or a point-in-time recovery — that's why you BEGIN before risky DML.

sql Lesson 21: INSERT, UPDATE, DELETE (These commands form the core Data Manipulation Language (DML) suite used to modify the actual records stored within your database tables. Understanding these operations is essential for maintaining data integrity and ensuring that the information retrieved by queries remains accurate and current. You reach for these operations whenever you need to add new entities, modify existing state, or remove obsolete information from your system.) — free Python cheat sheet